Homework Assignment 01
Submission deadline: February 14 (Frisday) by Midnight

You must submit your solution through blackboard. Your solution should be written in the space provided in this document. Once you have finished preparing the solution make sure to rename the doc file as: YourLastName_YourFirstName_HW01.doc.

Q01. Problem Statement:

Task 1: Write a program that reads a 2D array (matrix) of dimensions m x m from standard input (stdin). The elements should be read row by row.

Sample Method Structure:

```
private static int[][] readMatrix(int rows, int columns,
BufferedReader reader) throws IOException {
    int[][] matrix = new int[rows][columns];
    // Logic to read the matrix row by row

    // Your code should go here
    return matrix;
}
```

Challenge:

Task 2: Implement a method to read the matrix with optimized complexity, avoiding nested loops that significantly increase runtime for large matrices.

Your code should be here:
vvvv (the code is on the next page)

```java
package HW01;

import java.util.Scanner;

public class HW01_Q1_Task_2 {

	private static int[][] readMatrix(int rows, int columns, Scanner kb) {
	  int[][] matrix = new int[rows][columns];


	  int index = 0;
	for (int i = 0; i < rows * columns; i++) {
	  int rowIndex = index / columns;
	  int colIndex = index % columns;
	  matrix[rowIndex][colIndex] = kb.nextInt();
	  index++;
	}
	    return matrix;
	  }

	public static void main(String[] args) {

		int rows;
		int columns;
		Scanner kb = new Scanner(System.in);

		System.out.println("How many rows?");
		rows = kb.nextInt();
		System.out.println("How many columns?");
		columns = kb.nextInt();

		System.out.println("Enter the values for the matrix:");

		int[][] matrix = readMatrix(rows, columns, kb );

		System.out.println("The matrix:");
		for (int i = 0; i < rows * columns; i++) {
			int rowIndex = i / columns;
			int colIndex = i % columns;
			System.out.print(matrix[rowIndex][colIndex] + " ");
			if (colIndex == columns - 1)
				System.out.println();
```

```
            }
                kb.close();
        }
}
```

Q02. Problem Statement

Determine the processing time T(n) of the recursive algorithm:

```
1 int myTest( int n ) {
2 if ( n <= 0 ) return 0;
3  else {
4           int i = random( n - 1 );
5           return myTest( i ) + myTest( n - 1 - i );
6  }
7 }
```

providing the algorithm random(int n) spends one time unit to return a random integer value uniformly distributed in the range [0, n] whereas all other instructions spend a negligibly small time (e.g., T (0) = 0).

Hints: derive and solve the basic recurrence relating T(n) in average to T(n−1), ..., T(0). You might need the equation 1 + 1 +⋯+ 1 = n for deriving the explicit formula for T (n).

Your answer should be written here:

The best case of this recursive algorithm would be O(1). In this case the n would be less than or equal to 0 resulting in the if statement being true and the recursive method returning 0.

The average and worst case of this recursive algorithm would be O(n). This is determined by using T(n) = 1 + 2T(n/2) -> 1 + 2T(n/4) -> 1 + 2T(n/8) -> 1 + 2T(n/16). Although evetime the T(n) is being divided by two it is because the return statement has two recursive calls and doesn't split anything in half resulting in the complexity not being log n (base 2)

Q03. Problem Statement

Given an integer array A and an integer k, we want to find all pairs of elements in the array that add up to k. For each pair found, print the two numbers. If it is not possible to find any such combination, print "Not possible." The elements in A are unique. The array elements and k could be positive, zero, or negative. You do not need to print the other permutation that represents the same pair.

Sample Run 1

```
Enter size of array: 10
Enter the elements: 2 5 6 11 -1 0 25 -3 1 8
Enter the value of k: 22
-3 25
```

Sample Run 2

```
Enter size of array: 10
Enter the elements: 2 5 6 11 -1 0 25 -3 1 8
Enter the value of k: 50
Not possible.
```

There are two (to four) algorithms to solve this problem. One is a brute force algorithm, while the others are more efficient but require more code to implement. Design and analyze the big-O time complexity of two algorithms, then implement them.

```
public void findPairsBruteForce(int[] input, int target) {
}    /* TODO */
public void findPairsImproved(int[] input, int target) {
     mergeSort(input); // O(n log n) sort given to you
 // Efficiency should be better than the brute force algorithm! }
```

Your code should be written here:
The code is written below vvv

Explanation and Justification:

My approach for this question was to first make the main method so that the user can input the necessary values. Later in this method there is a call to both the findPairsBruteForce and findPairImproved. For the findPairsBruteForce method there is a nested for loop that gets the sum of each possible pair. This method won't add the same value to itself either because when initializing the value for j there is a "+ 1" which avoids the calculation of the same num twice. For the findPairImproved method my approach was to first apply the merge sort algorithm to the method findPairImproved in order to decrease the time complexity later on when the array is searched for the sum of two numbers. This is useful because with the array being sorted a while loop can be used to repeat a process rather than a nested for loop which has a time complexity of O(n).

```java
package HW01;

import java.util.Scanner;

public class HW01_Q3 {

	public static void findPairsBruteForce(int[] input, int target) {
		boolean noSolution = true;
	    for (int i = 0; i < input.length; i++) {
	        for (int j = i + 1; j < input.length; j++) {
	            if (input[i] + input[j] == target) {
	                System.out.println(input[i] + " " + input[j]);
	                noSolution = false;
	            }
	        }
	    }
	    if (noSolution) {
	        System.out.println("Not possible.");
	    }
	}


	public static void mergeSort(int[] input) {
	    if (input.length < 2) {
	        return;
	    }

	    int mid = input.length / 2;
	    int[] left = new int[mid];
	    int[] right = new int[input.length - mid];

	    for (int i = 0; i < mid; i++) {
	        left[i] = input[i];
	    }
	    for (int i = mid; i < input.length; i++) {
	        right[i - mid] = input[i];
	    }

	    mergeSort(left);
	    mergeSort(right);
	    merge(input, left, right);
	}
```

```
        public static void merge(int[] input, int[] leftHalf, int[] rightHalf) {
                int i = 0, j = 0, k = 0;

    while (i < leftHalf.length && j < rightHalf.length) {
        if (leftHalf[i] <= rightHalf[j]) {
                input[k++] = leftHalf[i++];
        } else {
                input[k++] = rightHalf[j++];
        }
    }

    while (i < leftHalf.length) {
        input[k++] = leftHalf[i++];
    }

    while (j < rightHalf.length) {
        input[k++] = rightHalf[j++];
    }
}


        public static void findPairsImproved(int[] input, int target) {
                mergeSort(input);

                int left = 0;
                int right = input.length - 1;
                boolean noSolution = true;

                while (left < right) {
                        int sum = input[left] + input[right];
                        if (sum == target) {
                                System.out.println(input[left] + " " + input[right]);
                                noSolution = false;
                                left++;
                                right--;
                        } else if (sum < target) {
                                left++;
                                } else {
                                        right--;
                                }
                }
            if (noSolution) {
```

```
                System.out.println("Not possible.");
            }
        }

        public static void main(String[] args) {
                int size;
                int target;
                Scanner kb = new Scanner(System.in);

                System.out.print("Enter size of array: ");
                size = kb.nextInt();

                int[] input = new int[size];
                System.out.print("Enter the elements: ");
                for(int i = 0; i<size; i++)
                        input[i] = kb.nextInt();

                System.out.print("Enter the value of k: ");
                target = kb.nextInt();

                System.out.println("\n" + "BruteForce solution");
                findPairsBruteForce(input, target);

                System.out.println("\n" + "Improved solution");
                findPairsImproved(input, target);

                kb.close();
        }

}
```

Rubrics guideline for grading:

Q01.

Correct Use of Nested Loops (40 points)
Task 1: Evaluates the appropriate and efficient use of nested loops for reading the matrix.
Correct Implementation of Optimized Method (40 points)
Task 2: Evaluates the innovative approach used to read the matrix with reduced complexity, avoiding nested loops.
Efficiency and Performance (20 points)
Assesses how the solution improves performance, particularly for larger matrices.

Q02.

Derivation of the Recurrence Relation (30 points)
- Correctly derives the basic recurrence relation for T(n).
- Shows a clear method for relating T(n) to T(n−1), ..., T(0).

Solving the Recurrence Relation (30 points)
- Successfully solves the derived recurrence relation.
- Presents a clear and logical step-by-step solution.

Accuracy of the Solution (30 points)
- Provides correct values or formula for T(n).

Explanation and Justification (10 points)
- Clearly explains each step of the solution.
- Justifies the approach and methods used in solving the recurrence.

Q03.

Implementation of Brute Force Algorithm (findPairsBruteForce) (30 points)
Implementation of Improved Algorithm (findPairsImproved) (40 points)
Explanation and Justification (10 points)
- Clearly explains each step of the solution.
- Justifies the approach and methods used in solving the recurrence.